Sesión 9. Persistencia y Offline First

Apuntes de estudio para estudiantes

Proyecto de referencia: Pokédex con PokeAPI, Kotlin, Compose, ViewModel, Flow y Koin. Estos apuntes explican la clase completa y complementan la guía práctica de la fase A.

Objetivo de aprendizaje

Persistir datos y mantener funciones útiles cuando la conexión es lenta, inestable o inexistente. Al terminar, debes poder explicar dónde guardar cada dato, cómo observarlo desde la interfaz y cómo actualizarlo sin perder información válida.

Resultados que debes poder demostrar

1. Consultar una lista de Pokémon después de cerrar el proceso y reabrir sin red.
2. Consultar una ficha previamente descargada, con tipos, medidas y estadísticas.
3. Guardar la vista compacta y recuperarla al iniciar de nuevo.
4. Actualizar el esquema sin borrar fichas existentes.
5. Mantener contenido visible cuando falla una actualización y explicar el alcance de la caché.

Ruta de lectura

Páginas	Tema
2-5	Persistencia, Entity, DAO, Database y mappers
6-9	Relaciones, transacciones, migraciones y DataStore
10-13	Fuente de verdad, caché, sincronización y estados de UI
14-16	Pruebas, laboratorio y tarea de favoritos
17-18	Autoevaluación, glosario y referencias

Los bloques de código son fragmentos de estudio: omiten package e imports cuando no son el tema. No reemplazan archivos completos salvo indicación expresa. La solución integrada del proyecto utiliza esquema v2.

Cómo estudiar: lee una sección, explica el concepto con tus palabras y sigue el recorrido de bulbasaur desde la API hasta la pantalla. Después prueba el caso sin internet.

1. Persistencia: qué sobrevive al cierre

Persistir significa escribir información en un almacenamiento que se puede volver a leer en una ejecución posterior. Una lista guardada solo en una propiedad del ViewModel continúa en memoria; no constituye una base local.

Elige el almacenamiento según el dato

Dato del proyecto	Lugar y motivo
Lista, fichas y estadísticas	Room: filas relacionadas y consultas SQL.
Vista compacta	DataStore: una preferencia pequeña.
Búsqueda y mensajes de carga	StateFlow del ViewModel: estado de la pantalla.
Imágenes	Caché de archivos del cargador de imágenes; Room conserva la URL.

Un ViewModel ayuda a mantener el estado durante recreaciones de la interfaz asociada, pero no garantiza que ese estado sobreviva a la muerte del proceso. StateFlow publica valores; su función no es escribirlos en disco. Tampoco una función suspend guarda datos por sí misma: debe ejecutar una operación de persistencia.

Tres situaciones diferentes

• Recomposición: Compose vuelve a dibujar una parte de la pantalla. No equivale a reiniciar la app.

• Recreación de la actividad: cambia la instancia de la pantalla; el ciclo de vida y los mecanismos de restauración determinan qué se conserva.

• Proceso terminado: se pierde la memoria del proceso. La prueba de persistencia debe comprobar una lectura posterior desde disco.

Aplicación a la Pokédex

Si HomeScreen vuelve a mostrar 30 Pokémon después de forzar la detención y abrir sin internet, existe evidencia de persistencia de la lista. Si solo alternas entre pantallas, podrías estar viendo datos que nunca abandonaron la memoria.

Sin una descarga previa puede no haber contenido local. Una experiencia offline correcta también contempla una base vacía y ofrece un mensaje claro con reintento.

Borrar los datos de la app elimina el almacenamiento local habitual. No lo uses como paso de la prueba que busca demostrar conservación de datos.

2. Room y el modelo de datos

Room organiza el acceso a una base SQLite mediante clases anotadas y genera parte del código durante la compilación. En esta clase sus tres piezas son Entity, DAO y Database. La primera describe datos; la segunda define operaciones; la tercera reúne el esquema y da acceso a los DAO.

Entity: una tabla con identidad estable

```
@Entity(tableName = "pokemon")
data class PokemonEntity(
    @PrimaryKey val id: Int,
    val name: String,
    val imageUrl: String
)
```

@Entity define la tabla pokemon. Cada instancia representa una fila; id, name e imageUrl son columnas. @PrimaryKey determina qué Pokémon identifica la fila. Usamos el id estable del recurso remoto para reconocerlo en descargas posteriores.

Por ejemplo, una fila puede contener id = 1 y name = "bulbasaur". Al refrescar, ese mismo id debe actualizar la fila existente. Si generáramos un identificador local diferente en cada descarga, sería más fácil producir duplicados.

La lista y la ficha no son lo mismo

Tabla	Qué representa
pokemon	Resumen usado por la lista: id, nombre y URL.
pokemon_details	Ficha descargada: medidas, tipos y fecha local.
pokemon_stats	Estadísticas vinculadas a la ficha.

Tener la fila de bulbasaur en pokemon no demuestra que se haya guardado su ficha. El endpoint de lista aporta resúmenes; la pantalla de detalle necesita descargar y persistir información adicional.

Entity, DTO y modelo de dominio tienen tareas distintas. El DTO representa la respuesta de PokeAPI, la Entity representa almacenamiento y el dominio representa los datos que usa la aplicación.

Lectura de apoyo: Entidades de Room.

3. DAO: lecturas, escrituras y consultas

Un DAO reúne las operaciones permitidas sobre los datos. Room genera su implementación. Las consultas usan los nombres reales de las tablas y columnas; parámetros como :name enlazan un argumento de la función.

```
@Dao
interface PokemonDao {
    @Query("SELECT * FROM pokemon ORDER BY id")
    fun observeAll(): Flow<List<PokemonEntity>>

    @Query("SELECT * FROM pokemon WHERE name = :name")
    suspend fun findByName(name: String): PokemonEntity?

    @Upsert
    suspend fun upsertAll(items: List<PokemonEntity>)
}
```

suspend y Flow resuelven necesidades distintas

suspend permite esperar una operación sin bloquear el hilo mediante una espera convencional. findByName entrega una lectura puntual: una fila o null. upsertAll finaliza después de escribir. Flow representa una lectura observable: el consumidor recibe resultados y nuevas emisiones cuando Room detecta cambios pertinentes. No agregamos suspend a observeAll solo por devolver un flujo.

Marcar una función suspend no mueve automáticamente todo su código a otro hilo. Room ofrece ejecución asíncrona para estas operaciones. Evita consultas bloqueantes en el hilo principal y no uses allowMainThreadQueries como solución de producción.

Las lecturas observables se recolectan con collect. Una invalidación de tabla puede volver a ejecutar la consulta aunque su resultado lógico no cambie; distinctUntilChanged ayuda a evitar emisiones equivalentes hacia la capa siguiente.

Búsqueda SQL: ejemplo adicional

```
@Query("""SELECT * FROM pokemon
    WHERE name LIKE '%' || :query || '%'
    ORDER BY id""")
fun search(query: String): Flow<List<PokemonEntity>>
```

Esta consulta es una alternativa para filtrar en la base. El proyecto actual filtra en el ViewModel la lista ya cargada. Ambos enfoques buscan únicamente entre los resúmenes disponibles localmente.

Upsert inserta una fila ausente o actualiza la existente por su clave. Repetir la misma página no debería multiplicar filas. No equivale a borrar y volver a crear toda la base.

Lectura de apoyo: DAO y consultas asíncronas.

4. Database, mappers e inyección

PokemonDatabase declara las entidades que forman parte del esquema. La versión identifica la estructura local; no representa la versión de PokeAPI ni el número de registros descargados.

```
@Database(
    entities = [PokemonEntity::class,
        PokemonDetailEntity::class, PokemonStatEntity::class],
    version = 2, exportSchema = true
)
abstract class PokemonDatabase : RoomDatabase() {
    abstract fun pokemonDao(): PokemonDao
    abstract fun detailDao(): PokemonDetailDao
}
```

Una instancia compartida de la base

```
// Fragmento de dataModule, dentro de module { ... }
single {
    Room.databaseBuilder(androidContext(),
        PokemonDatabase::class.java, "pokedex.db")
        .addMigrations(MIGRATION_1_2)
        .build()
}
single<PokemonRepository> {
    PokemonRepositoryImpl(get(), get())
}
```

Koin resuelve PokeApi y PokemonDatabase para construir el repositorio. Compartir la base evita crear conexiones y observadores innecesarios por pantalla. Los DAO quedan en data; los casos de uso reciben el contrato de dominio del repositorio.

Mappers: convertir sin mezclar responsabilidades

```
fun PokemonEntity.toDomain() = Pokemon(id, name, imageUrl)
fun Pokemon.toEntity() = PokemonEntity(id, name, imageUrl)
```

El proyecto también transforma la ficha remota en entidades y la lectura relacionada en PokemonDetail. Conserva heightDm y weightHg en las unidades del servicio; para mostrar metros y kilogramos divide entre 10f. Así evita confundir valores almacenados con unidades de presentación.

KSP genera el código de Room al compilar. exportSchema guarda una descripción del esquema que sirve para revisar cambios y probar migraciones; no contiene las filas de cada usuario.

Las dependencias y versiones se mantienen en la configuración del proyecto. Para incorporar Room a la base de sesión 8, utiliza los pasos específicos de la guía de fase A.

5. Relaciones: fichas y estadísticas

En el esquema del laboratorio, un resumen puede tener cero o una ficha descargada. Una ficha puede tener varias estadísticas. Cada estadística pertenece a una ficha identificada por pokemonId. Esta segunda relación es de uno a muchos.

Qué representa cada clave

La clave primaria de pokemon_details es pokemonId y referencia pokemon.id. En pokemon_stats la clave es compuesta: pokemonId y name. Por eso hp de bulbasaur y hp de otro Pokémon son filas distintas, pero la misma pareja no se repite.

```
@Entity(tableName = "pokemon_stats",
    primaryKeys = ["pokemonId", "name"],
    foreignKeys = [ForeignKey(
        entity = PokemonDetailEntity::class,
        parentColumns = ["pokemonId"],
        childColumns = ["pokemonId"],
        onDelete = ForeignKey.CASCADE
    )], indices = [Index("pokemonId")])
data class PokemonStatEntity(
    val pokemonId: Int,
    val name: String,
    val baseValue: Int,
    val position: Int
)
```

ForeignKey protege la referencia: no se debe guardar una estadística de una ficha inexistente. CASCADE elimina sus estadísticas si se elimina la ficha. Index ayuda al acceso por pokemonId. position permite reconstruir el orden de las estadísticas al mapear el resultado.

Consultar una relación no es definir una restricción

@Relation ayuda a Room a reunir objetos relacionados en una lectura. No crea por sí sola una clave foránea. @Embedded incorpora las columnas de un objeto al resultado de consulta. Estas anotaciones se usan en CachedPokemonDetail, que no es una tabla adicional.

Los tipos se guardan como JSON en typesJson para simplificar el ejercicio. Es adecuado para reconstruir la lista al leer una ficha. Si se necesitaran consultas y relaciones frecuentes por tipo, convendría evaluar tablas específicas.

Lectura de apoyo: Relaciones uno a muchos.

6. Transacciones: leer y guardar conjuntos

Una ficha debe mantenerse coherente con sus estadísticas. Si guardamos la ficha nueva y el proceso falla antes de guardar sus estadísticas, podríamos mostrar una combinación incompleta. Una transacción agrupa cambios locales: se confirman juntos o se deshacen si falla la operación.

Lectura relacionada del proyecto

```
data class CachedPokemonDetail(
    @Embedded val detail: PokemonDetailEntity,
    @Relation(parentColumn = "pokemonId", entityColumn = "id")
    val pokemon: PokemonEntity,
    @Relation(parentColumn = "pokemonId", entityColumn = "pokemonId")
    val stats: List<PokemonStatEntity>
)

// En PokemonDetailDao
@Transaction
@Query("""SELECT * FROM pokemon_details WHERE pokemonId =
    (SELECT id FROM pokemon WHERE name = :name)""")
fun observe(name: String): Flow<CachedPokemonDetail?>
```

La consulta principal parte de pokemon_details. Si solo existe el resumen, el resultado es null. @Transaction permite obtener una vista coherente cuando Room necesita varias consultas para reconstruir las relaciones.

Escritura atómica de una ficha

```
// dto ya fue descargado; summary, detail y stats ya se mapearon.
db.withTransaction {
    db.pokemonDao().upsertAll(listOf(summary))
    db.detailDao().upsert(detail)
    db.detailDao().deleteStats(dto.id)
    db.detailDao().insertStats(stats)
}
```

El orden satisface las referencias: resumen, ficha y estadísticas. Reemplazar el conjunto de estadísticas evita conservar una fila que ya no esté en la nueva respuesta. Al estar dentro de la transacción, un fallo permite recuperar el estado previo en lugar de dejar un borrado parcial.

> La llamada HTTP queda fuera de la transacción. Una transacción de Room protege operaciones de la base local; no vuelve atómica una operación entre internet y SQLite.

7. Migraciones: cambiar sin borrar datos

Una migración transforma una base instalada desde una versión de esquema hacia otra. En la solución integrada, v1 ya incluye lista, fichas y estadísticas. v2 añade lastUpdated a pokemon_details para registrar la fecha de una descarga válida.

```
// Campo incorporado a PokemonDetailEntity en v2
@ColumnInfo(defaultValue = "0")
val lastUpdated: Long = 0L

val MIGRATION_1_2 = object : Migration(1, 2) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL("ALTER TABLE pokemon_details ADD COLUMN " +
            "lastUpdated INTEGER NOT NULL DEFAULT 0")
    }
}
```

INTEGER almacena el Long y NOT NULL exige un valor para todas las filas. DEFAULT 0 permite conservar las fichas anteriores, cuya fecha de descarga no se conoce. La anotación ColumnInfo mantiene el valor predeterminado del esquema alineado con el SQL. El valor Kotlin 0L también permite crear objetos sin pasar ese argumento.

Pasos que forman el cambio completo

1. Modificar la Entity y subir @Database a version = 2.

2. Registrar MIGRATION_1_2 con addMigrations en el builder.

3. Generar y conservar los esquemas JSON de ambas versiones.

4. Crear una base v1 con datos, migrarla y validar estructura y contenido.

Una instalación limpia de v2 no ejecuta el recorrido de una base v1 existente. Por eso compilar, desinstalar y reinstalar no demuestra que una migración funcione. Las migraciones automáticas pueden cubrir cambios compatibles; los cambios ambiguos o complejos requieren instrucciones adicionales o una migración manual.

> La guía independiente de fase A usa una v1 simplificada que solo tiene pokemon. Esa base no equivale a la v1 del laboratorio completo. Al incorporar fichas sobre una instalación de fase A, hay que versionar y migrar su esquema real.

Evita una recreación destructiva para ocultar errores de migración: borrar la base invalida el objetivo de conservar datos del estudiante o del usuario.

Lectura de apoyo: Migraciones de Room.

8. DataStore: preferencias de la pantalla

La vista compacta es una elección del usuario, no una fila del catálogo remoto. Preferences DataStore permite escribirla con una clave y observarla como Flow. En la clase se usa un Boolean: false representa la vista normal cuando todavía no hay un valor guardado.

```
val Context.settings by preferencesDataStore(name = "settings")

class SettingsRepositoryImpl(
    private val store: DataStore<Preferences>
) : SettingsRepository {
    override val compact = store.data
        .catch { error ->
            if (error is IOException) emit(emptyPreferences())
            else throw error
        }
        .map { it[COMPACT] ?: false }
        .distinctUntilChanged()

    override suspend fun setCompact(value: Boolean) {
        store.edit { it[COMPACT] = value }
    }

    private companion object {
        val COMPACT = booleanPreferencesKey("compact_view")
    }
}
```

El delegado se declara una vez en el nivel superior. Koin inyecta androidContext().settings en la implementación. Se comparte una instancia por archivo y proceso. El bloque edit actualiza la preferencia de forma transaccional dentro de DataStore; no forma una transacción conjunta con Room.

Lectura, escritura y errores

El ViewModel observa compact y publica su valor en el estado de pantalla. Al cambiar el control llama a setCompact; DataStore escribe y emite el valor actualizado. El proyecto también expone el progreso de guardado y los errores de configuración.

En esta preferencia no crítica, una IOException de lectura emite preferencias vacías y produce la vista normal. Eso es una política de recuperación, no la prueba de que el archivo se reparó o que el flujo reintentará automáticamente. Los demás errores se propagan.

Preferences DataStore usa claves tipadas sin definir un esquema de objetos propio. Proto DataStore usa mensajes con un esquema explícito. Para el catálogo con relaciones y consultas parciales seguimos usando Room.

Lectura de apoyo: DataStore.

9. Single Source of Truth en la Pokédex

La fuente de verdad define desde dónde obtiene cada consumidor el estado autorizado de un dato. En esta arquitectura, la pantalla lee el catálogo desde Room. PokeAPI aporta respuestas que el repositorio transforma y guarda antes de que la UI vea el cambio.

Dos recorridos con tareas diferentes

```
LECTURA
Room -> DAO Flow -> repositorio -> caso Get -> ViewModel -> UI

ACTUALIZACIÓN
UI -> caso Refresh -> repositorio -> PokeAPI
   -> mappers -> Room -> nueva emisión del recorrido de lectura
```

Estos recorridos evitan que la pantalla mezcle una lista directa de la API con otra lista local. Una respuesta de refresh no se asigna por separado a la UI: la actualización de Room es la que termina produciendo el contenido visible.

Contrato de la solución integrada

```
interface PokemonRepository {
    fun observePokemonList(): Flow<List<Pokemon>>
    fun observePokemonDetail(name: String): Flow<PokemonDetail?>
    suspend fun refreshPokemonList()
    suspend fun refreshPokemonDetail(name: String)
}
```

Los casos Get representan las lecturas observables. Los casos Refresh ejecutan la descarga y escritura. En fase A solo se adapta la lista; al completar la fase B, el detalle también se observa localmente. El tipo nullable expresa una ficha aún ausente.

Qué resuelve el repositorio

Decide cómo combinar las fuentes, mantiene las conversiones en data y ofrece al dominio operaciones con significado. En el proyecto, observePokemonList transforma Entity en Pokemon; refreshPokemonList descarga 30 resúmenes y aplica Upsert. Los ViewModels no necesitan conocer el nombre del archivo de base ni ejecutar SQL.

> La fuente de verdad es una decisión por tipo de dato. Room gobierna la lectura del catálogo local; DataStore gobierna la vista compacta. La autoridad remota del catálogo sigue siendo PokeAPI.

Lectura de apoyo: Arquitectura offline-first.

10. Caché: alcance, antigüedad y límites

La caché del catálogo conserva una copia local de información descargada. Permite volver a usarla sin repetir inmediatamente una petición, pero su contenido depende de lo que la app haya solicitado y guardado. No significa que toda PokeAPI esté disponible offline.

Tres capas que debes comprobar por separado

Capa	Qué hay guardado	Qué no garantiza
Lista	Resúmenes de las páginas solicitadas.	Fichas completas de todos los resultados.
Detalle	Ficha y estadísticas ya descargadas.	Una ficha que nunca se abrió o precargó.
Imagen	Bytes conservados por su caché de archivos.	Que imageUrl por sí sola pueda dibujarla sin red.

En el laboratorio, la lista solicita limit = 30 y offset = 0. Buscar localmente entre esos datos no consulta todo el catálogo. Una ampliación con paginación debe guardar y consultar las páginas disponibles; no debe borrar todo el contenido porque una página nueva esté vacía.

Comparar estrategias de lectura

Cache-first: reutiliza lo guardado y solicita la red cuando falta, según su política. Network-first: intenta la red y recurre a la copia local si falla; puede hacer esperar al usuario. Lectura local con refresh: muestra la copia disponible y actualiza la base por separado. Este último recorrido es el usado en la clase y permite conservar contenido durante la actualización.

lastUpdated y la política de frescura

El mapper de detalle guarda System.currentTimeMillis() después de obtener la respuesta. Es una fecha local en milisegundos, no la fecha oficial de modificación del Pokémon en el servidor. El valor 0 significa que la fecha histórica se desconoce.

Una política futura podría calcular la edad del dato y decidir si corresponde actualizar. La implementación de clase refresca al entrar y por reintento manual: tener lastUpdated no implementa automáticamente un TTL ni una descarga periódica. Si el reloj del dispositivo cambia, también cambia la interpretación de esa edad.

Conservar contenido durante un fallo

La app primero obtiene y mapea la respuesta; después escribe. Si la red falla antes, no elimina la última copia válida. Un dato anterior puede seguir siendo útil, y la interfaz puede informar que no logró actualizarlo. Nunca se debe presentar una descarga fallida como una confirmación de frescura.

La caché de archivos puede evictar imágenes. Usa un placeholder y verifica el caso sin bytes locales. Persistencia de texto y disponibilidad de imágenes requieren comprobaciones diferentes.

11. Sincronización y trabajo pendiente

Aquí sincronizar el catálogo significa descargar el estado remoto y actualizar su copia local. La lista usa GET /pokemon?limit=30&offset=0; la ficha usa GET /pokemon/{name}. La API pública documentada por PokeAPI ofrece lectura y no recibe los favoritos ni la configuración del usuario.

Secuencia del refresh de una ficha

1. Solicitar la ficha por nombre y esperar una respuesta válida.

2. Mapear resumen, ficha, tipos y estadísticas antes de escribir.

3. Guardar las tablas relacionadas en una transacción local.

4. Permitir que el Flow publique el nuevo contenido y finalizar el indicador de refresh.

El repositorio del proyecto usa Mutex para serializar los refresh de lista y los de detalle dentro de su instancia. Esto reduce la aplicación de respuestas fuera de orden entre llamadas protegidas. Un Mutex vive en memoria: no es una cola persistente ni sincroniza dispositivos.

WorkManager como ampliación

Cuando se requiere trabajo que se programe más allá de la pantalla, WorkManager permite mantener solicitudes de trabajo y aplicar restricciones, como disponer de red. Los errores temporales pueden producir reintentos con backoff. El sistema decide cuándo ejecutar el trabajo; no se promete una hora exacta ni que una red disponible garantice que el servidor responda.

¿Y si existiera un backend de favoritos?

Sería una funcionalidad adicional. Una estrategia posible guarda el favorito local y una operación pendiente en la misma transacción Room. Un worker envía esa operación cuando puede. Un identificador estable ayuda al servidor a reconocer reintentos y evitar efectos duplicados. La política de conflictos debe especificar qué ocurre si dos dispositivos editan el mismo dato.

La idempotencia debe abarcar la operación y su interpretación en el servidor; no se consigue solo porque el cliente genere un UUID. PokeAPI no ofrece ese backend del usuario. En la tarea los favoritos permanecen locales.

Lectura de apoyo: Contrato y endpoints de PokeAPI.

Lectura de apoyo: Trabajo persistente con WorkManager.

12. ViewModel y estados de interfaz

El ViewModel coordina observación y acciones. Inicia un collect del caso Get para recibir Room y solicita un refresh por separado. Un problema al actualizar no debe reemplazar una ficha válida por una pantalla vacía.

Estados que tienen significados distintos

Estado	Interpretación en la clase
isLoading	La lectura local inicial todavía no terminó.
isRefreshing	Hay una actualización remota en curso.
pokemon / detail	Contenido local que se puede mostrar.
refreshError	No se logró actualizar; puede existir contenido útil.
error	Problema al leer el almacenamiento local.
settingsError	Problema de lectura o guardado de la configuración.

La primera emisión de Room puede ser una lista vacía y aun así completar la lectura inicial. Si la descarga sigue en curso, la UI puede mostrar “Descargando Pokémon”. Si no hay contenido y el intento falla, debe ofrecer una explicación y reintento.

Patrón de manejo de refresh

```
// Dentro de una coroutine del ViewModel; fragmento didáctico.
try {
    refreshPokemonList()
} catch (cancelled: CancellationException) {
    throw cancelled
} catch (_: Exception) {
    _state.update {
        it.copy(refreshError = "No se pudo actualizar.")
    }
} finally {
    _state.update { it.copy(isRefreshing = false) }
}
```

Antes del bloque se marca isRefreshing = true y se limpia el mensaje de refresh. En el catch se conserva la lista. La cancelación se propaga para respetar el ciclo de vida. Los errores de lectura local y preferencias se gestionan en sus propios recorridos.

Con caché y error de red, muestra contenido y aviso juntos. Sin ficha local, muestra “Detalle no descargado”. Diferencia una búsqueda sin coincidencias de una base sin Pokémon.

Compose observa el estado con collectAsStateWithLifecycle. No inicies una petición nueva en cada recomposición: las acciones de actualización deben tener un punto de entrada controlado.

13. Pruebas y diagnóstico

Una compilación verifica contratos y generación de código; no demuestra que una app pueda reabrir sin red. La verificación debe combinar pruebas de comportamiento con inspección de los datos guardados.

Recorrido manual reproducible

1. Con internet, descargar la lista y abrir bulbasaur hasta ver medidas y estadísticas.
2. Activar modo avión y desactivar Wi-Fi si sigue habilitado.
3. Forzar la detención desde Ajustes y volver a abrir desde el lanzador, sin borrar datos.
4. Comprobar lista, búsqueda, ficha guardada y preferencia compacta.
5. Intentar una ficha nunca guardada y un refresh sin red; verificar mensajes y conservación del contenido.
6. Restablecer la red y actualizar: los datos se recuperan sin duplicar filas.

Database Inspector

```
SELECT COUNT(*) FROM pokemon;
SELECT * FROM pokemon_details WHERE pokemonId = 1;
SELECT name, baseValue FROM pokemon_stats
WHERE pokemonId = 1 ORDER BY position;
```

El conteo esperado de fase A es 30 en una base limpia. En una app ampliada, abrir o descargar fichas fuera de esa página puede agregar resúmenes adicionales: el conteo debe interpretarse según las operaciones realizadas.

Qué automatizar

Una prueba de DAO verifica Upsert y relaciones. Una prueba de repositorio simula una respuesta válida y otra fallida para comprobar que los datos previos permanecen. Una prueba del ViewModel verifica que el aviso de refresh no vacía el contenido. Para migraciones, crea v1 con filas conocidas, ejecuta la migración con MigrationTestHelper y comprueba el esquema y las filas resultantes.

Una base en memoria es útil para operaciones de DAO, pero no demuestra reapertura desde disco. La prueba de persistencia debe cerrar y volver a abrir el mismo almacenamiento, o reiniciar realmente el proceso con los datos preservados.

> Si se queda en el ícono de arranque, revisa Logcat, excepciones de inicio, inicialización y el estado del emulador. El splash por sí solo no identifica un error de Room ni de conectividad.

Lectura de apoyo: Pruebas y depuración de bases Room.

14. Laboratorio: fases y comprobaciones

El laboratorio adapta la Pokédex existente. Conserva Retrofit, navegación y modelos de dominio cuando el cambio no requiera modificarlos. Cambia los contratos de lectura y sus consumidores de forma coordinada.

Fase A. Lista local y Flow

Crear PokemonEntity, PokemonDao y PokemonDatabase; mapear filas y dominio; separar observePokemonList de refreshPokemonList; adaptar GetPokemonListUseCase, ViewModel y registro de Koin. El detalle puede conservar la consulta remota en esta fase independiente.

Evidencia: descargar 30 resúmenes, forzar la detención y reabrir sin internet. Lista y búsqueda funcionan. Un refresh fallido conserva la lista. La guía de fase A contiene las instrucciones y fragmentos completos para este paso.

Fase B. Ficha y estadísticas

Agregar entidades de detalle y estadísticas, la lectura con @Relation y el guardado con withTransaction. Adaptar el contrato de detalle a Flow<PokemonDetail?>, su caso Get, el caso Refresh, ViewModel y Koin. En una base instalada, incorporar tablas exige una migración desde el esquema que realmente tenga esa base.

Evidencia: una ficha descargada muestra medidas, tipos y estadísticas offline; una ficha ausente muestra su estado específico, sin inventar datos ni confundirla con un error de renderizado.

Fase C. Preferencia y evolución del esquema

Guardar la vista compacta con DataStore y observarla en HomeScreen. En el recorrido del laboratorio integrado, partir de su esquema v1 con tres tablas y migrar a v2 añadiendo lastUpdated. Probar la conservación de fichas y estadísticas.

Evidencia: preferencia recuperada después de reabrir y datos presentes tras la migración. Las fichas previas a v2 tienen lastUpdated = 0 hasta una actualización posterior.

Errores frecuentes al conectar las capas

Síntoma	Revisión útil
Se esperaba List y llega Flow	Actualizar Get y recolectar el resultado.
Koin no construye el ViewModel	Alinear constructor, registro y casos de uso.
Hay filas pero no se ven	Revisar si una rama de error oculta el contenido.
Lista offline; ficha ausente	Comprobar la descarga y la tabla de detalle.

15. Tarea: favoritos disponibles offline

Amplía la solución integrada de la clase para agregar y quitar favoritos desde el detalle. Incorpora “Solo favoritos” en HomeScreen. La selección debe sobrevivir al cierre del proceso y a un refresh de PokeAPI.

Diseño que debes justificar

Guarda los favoritos en una tabla Room separada de las filas que se reconstruyen desde la API. La identidad del favorito debe vincularse a un id estable del Pokémon. Elige una consulta o combinación de flujos que entregue los resúmenes marcados y permita volver a mostrar todos.

Una fila de favorito puede identificar al Pokémon y, si lo necesitas, registrar la fecha de selección. Decide si usarás una clave foránea y qué sucedería al eliminar caché: una política CASCADE mal elegida podría borrar una elección del usuario. Explica el comportamiento que adoptaste.

Migración y comportamiento esperado

1. Partir de la solución integrada v2 y crear v3 con la nueva tabla.
2. Migrar sin perder fichas, estadísticas ni fecha de actualización.
3. Marcar, desmarcar y filtrar sin depender de internet.
4. Reabrir offline y conservar la selección.
5. Refrescar el catálogo y mantener los favoritos.

La caché remota puede reconstruirse. Los favoritos son datos creados por el usuario. Evita que reemplazar o limpiar el catálogo los elimine accidentalmente. Esta tarea no requiere subirlos a PokeAPI.

Entrega y criterios

Entrega el proyecto, una explicación breve de la estructura y un video de hasta dos minutos que muestre marcado, filtro, modo avión, reinicio y refresh. Incluye una prueba de migración y una de persistencia.

Criterio	Puntos
Agregar y quitar favoritos	3
Filtro Solo favoritos	2
Conservación al reiniciar	2
Conservación ante refresh	2
Migración sin pérdida	1

La nota se basa en el comportamiento demostrado y en la coherencia del almacenamiento. Una captura de una lista marcada sin reiniciar no basta para verificar persistencia.

16. Autoevaluación con respuestas

Intenta responder primero sin mirar la explicación. Cada respuesta debe poder relacionarse con un archivo o una prueba del proyecto.

1. ¿Por qué StateFlow no reemplaza a Room?

StateFlow publica estado en memoria. Room escribe y consulta datos locales que pueden leerse en otra ejecución. El ViewModel puede reconstruir su StateFlow a partir del Flow del DAO.

2. ¿Qué diferencia hay entre suspend y Flow?

Una operación suspend permite esperar su resultado sin bloquear mediante una espera convencional. Flow permite recolectar una secuencia de emisiones. En el DAO usamos suspend para una escritura o lectura puntual y Flow para observar datos.

3. ¿@Relation crea una clave foránea?

No. Agrupa una lectura de datos relacionados. ForeignKey define la restricción de integridad; @Transaction mantiene coherencia cuando esa lectura requiere varias consultas.

4. ¿Por qué la ficha puede faltar si el nombre está en la lista?

La página de lista guarda resúmenes. El detalle y sus estadísticas se descargan mediante otra operación. Sin una ficha persistida, se muestra que todavía no se descargó.

5. ¿Qué demuestra que una migración funciona?

Que una base antigua con datos conocidos se transforma al esquema nuevo y conserva esos datos. Una instalación limpia solo comprueba la creación del esquema actual.

6. ¿Qué debe ocurrir si falla el refresh con caché disponible?

Debe terminar el indicador de actualización, mostrarse el aviso y conservarse el contenido local. No debe vaciarse la lista ni afirmarse que acaba de actualizarse.

7. ¿Por qué imageUrl no garantiza la imagen offline?

Es una dirección. Sin bytes locales y una política adecuada de caché, el cargador no tiene de dónde obtener la imagen al perder conectividad.

8. ¿PokeAPI sincroniza los favoritos del usuario?

No. En esta clase son datos locales. Sincronizarlos entre dispositivos requiere otro backend, autenticación y reglas para reintentos y conflictos.

9. ¿lastUpdated provoca un refresh automático?

No. Es información para decidir o comunicar frescura. La app necesita una política y un mecanismo explícitos para actualizar.

17. Glosario y referencias de estudio

Término	Significado en esta clase
Persistencia	Escritura y recuperación de datos entre ejecuciones.
Entity / DAO / Database	Tabla representada en Kotlin / operaciones / esquema y acceso.
DTO / mapper	Respuesta del servicio / conversión entre representaciones.
Upsert	Inserción o actualización según la clave existente.
Clave foránea	Referencia con una restricción de integridad.
Transacción	Conjunto de operaciones locales que se confirma o revierte junto.
Migración	Transformación entre versiones del esquema instalado.
SSOT	Fuente autorizada que consumen los lectores de un tipo de dato.
Caché / frescura	Copia reutilizable / criterio sobre su antigüedad aceptable.
Refresh	Descarga y guardado de una respuesta remota nueva.
Idempotencia	Repetir una operación mantiene su efecto previsto sin duplicarlo.

Documentación oficial

1. Room: visión general
2. Entidades y esquema
3. DAO y acceso a datos
4. Consultas asíncronas y Flow
5. Relaciones uno a muchos
6. Migraciones
7. Preferences y Proto DataStore
8. Arquitectura offline-first
9. Pruebas de bases de datos
10. Trabajo persistente con WorkManager
11. PokeAPI: contrato de lectura

Ejemplos y nombres contrastados con el proyecto Android de la clase: data/local, data/mapper, data/repository, data/preferences, domain y presentation. Alcance de estos apuntes: sesión 9, fases A-C y tarea local de favoritos.